



**Faculty of Computers and Artificial
Intelligence | Cairo University**



Introduction to Database Systems

Project Phase 3 | Physical ERD

Project 12: The Regional Farm-to-Table Distribution

TA: Lamiaa Atef

Section: S 33 - 34

Live Website URL: <https://db.yehia.dev>

NAME	ID
Yehia Hassan Abdelmoaty Hassan	20242447
Marina Emad Sobhy	20240782
Jana Ahmed Farahat Hassan	20240759
Noha Mohamed Ahmed	20240794
Hana Khaled Abdelhamed	20240650
Sohila Maher Shebat-Elhamd	20242174

Project 12: The Regional Farm-to-Table Distribution

System Description:

The Regional Farm-to-Table Distribution system is a cooperative network that connects local farmers directly with restaurants to ensure the delivery of fresh, seasonal produce. The system tracks farm inventories (harvest batches), restaurant orders, and the logistical delivery trips facilitated by drivers using refrigerated trucks.

Functional Requirements:

Req ID	Module / Feature	Requirement Description
FR-01	User Management	The system shall allow the registration and management of profiles for Local Farms, capturing their location and specialized crop types.
FR-02	User Management	The system shall allow the registration of Restaurants, capturing their delivery addresses and preferred delivery time windows.
FR-03	Inventory Management	Farmers shall be able to list new Harvest Batches, specifying the crop type, harvest date, and available quantity.
FR-04	Inventory Management	The system shall track the seasonal inventory and monitor the "freshness window" of harvest batches from harvest to delivery.

Req ID	Module / Feature	Requirement Description
FR-05	Order Processing	Restaurants shall be able to place Orders by selecting specific harvest batches from various farms and defining the quantity ordered.
FR-06	Logistics & Transport	The system shall allow the coordination of Delivery Trips, assigning specific drivers to routes to transport orders.
FR-07	Logistics & Transport	The system shall log the details of every delivery trip, including the driver responsible, the route taken, trip date, and total distance traveled.
FR-08	Reporting & Analytics	The system shall calculate and retrieve the crop type with the maximum number of orders placed by restaurants.
FR-09	Reporting & Analytics	The system shall identify farms that had no harvest batches listed or sold during the last month.
FR-10	Reporting & Analytics	The system shall identify the driver who completed the highest number of delivery trips in the last month.
FR-11	Reporting & Analytics	The system shall identify restaurants that did not place any produce orders in the last month.
FR-12	Reporting & Analytics	The system shall retrieve the specific harvest batches delivered to each restaurant in the last month.
FR-13	Reporting & Analytics	The system shall calculate the total revenue generated from sold batches for each farm.

Conceptual Database Model:

3.1 Entities and Attributes:

1. Farm

- Farm_ID (PK)
- Name
- Location

2. CropType

- CropTypeID (PK)
- Crop_Type

3. Harvest_Batch

- Batch_ID (PK)
- Harvest_Date
- Available_Quantity
- PricePerKG
- IsAvailable
- Farm_ID (FK)
- Crop_ID (FK)

4. Restaurant

- Restaurant_ID (PK)
- Name
- Delivery_Address (Comp: DeliveryAddress, City, PostalCode)
- ContactPhone
- Preferred_Delivery_Window

5. Order

- Order_ID (PK)
- Order_Date
- Restaurant_ID (FK)
- Status

6. Driver

- Driver_ID (PK)
- Name (Comp: First_Name, Last_Name)
- Phone

7. Trips

- Trip_ID (PK)
- Trip_Date
- Total_Distance
- Driver_ID (FK)

3.2 Relationships & Cardinalities:

1. **Farm** *Specializes_In* **CropType**.

- Relation Type: Many-to-Many (M:N)
- Description: A farm can specialize in multiple crop types, and a crop type can be grown by multiple farms. (Requires a bridge table in Phase 3).

2. **Farm** *Produces* **Harvest_Batch**.

- Relation Type: One-to-Many (1:M)
- Description: A farm can list multiple harvest batches, and a specific batch belongs to exactly one farm.

3. **CropType** *Is_Of_Type* **Harvest_Batch**.

- Relation Type: One-to-Many (1:M)
- Description: A crop type can categorize many harvest batches, and a specific harvest batch consists of exactly one crop type.

4. **Restaurant** *Places_Order* **Order**.

- Relation Type: One-to-Many (1:M)
- Description: A restaurant can place multiple orders over time, and an order is placed by exactly one restaurant.

5. **Order** *Includes* **Harvest_Batch**.

- Relation Type: Many-to-Many (M:N)
- Attribute: Quantity_Ordered.
- Description: An order can include multiple different harvest batches, and a single harvest batch can be sold across multiple different orders. (Requires an OrderDetails bridge table in Phase 3).

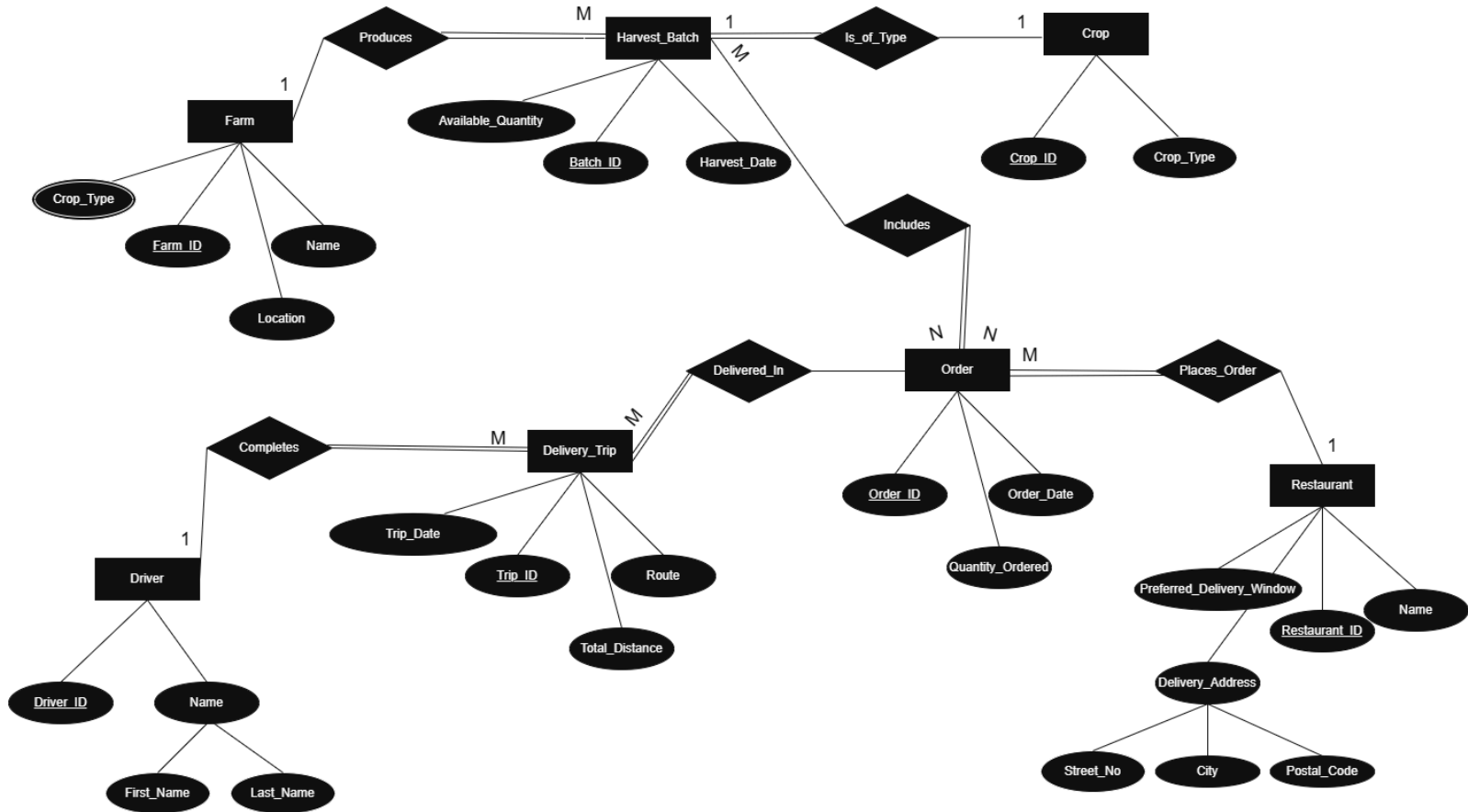
6. **Trips** *Carries* **Order**.

- Relation Type: Many-to-Many (M:N)
- Description: A trip can carry multiple orders, and a single large order might be split across multiple trips. (Requires a TripOrders bridge table in Phase 3).

7. **Driver** *Completes* **Trips**.

- Relation Type: One-to-Many (1:M)
- Description: A driver can complete multiple trips, and a single trip is assigned to exactly one driver.

4. Conceptual Entity Relationship Diagram (ERD)



5. Relational Mapping & Physical Transformations:

To transition from the Conceptual Entity Relationship Diagram (ERD) to the implemented Physical Database Schema on MS SQL Server, several structural transformations were applied to ensure data integrity, normalization, and relational compliance.

1. Resolution of Many-to-Many (M:N) Relationships Relational databases cannot directly implement M:N relationships. Therefore, associative entities (bridge tables) were created to break these down into corresponding One-to-Many (1:M) relationships:

- **OrderDetails:** Created to resolve the M:N relationship between Order and Harvest_Batch. A single order can contain multiple batches, and a batch can be distributed across multiple orders. This bridge table utilizes a composite primary key (OrderID, BatchID) and stores the intersection attribute Quantity_Ordered.
- **TripOrders:** Created to handle the M:N relationship between Delivery_Trip and Order. Since a delivery trip carries multiple orders, and a large order might theoretically span multiple trips, this table links TripID and OrderID.
- **FarmCropSpecialties:** Formed to resolve the M:N relationship between Farm and Crop. It connects FarmID and CropID to track the specific types of produce each local farm specializes in growing.

2. Key Assignments and Referential Integrity

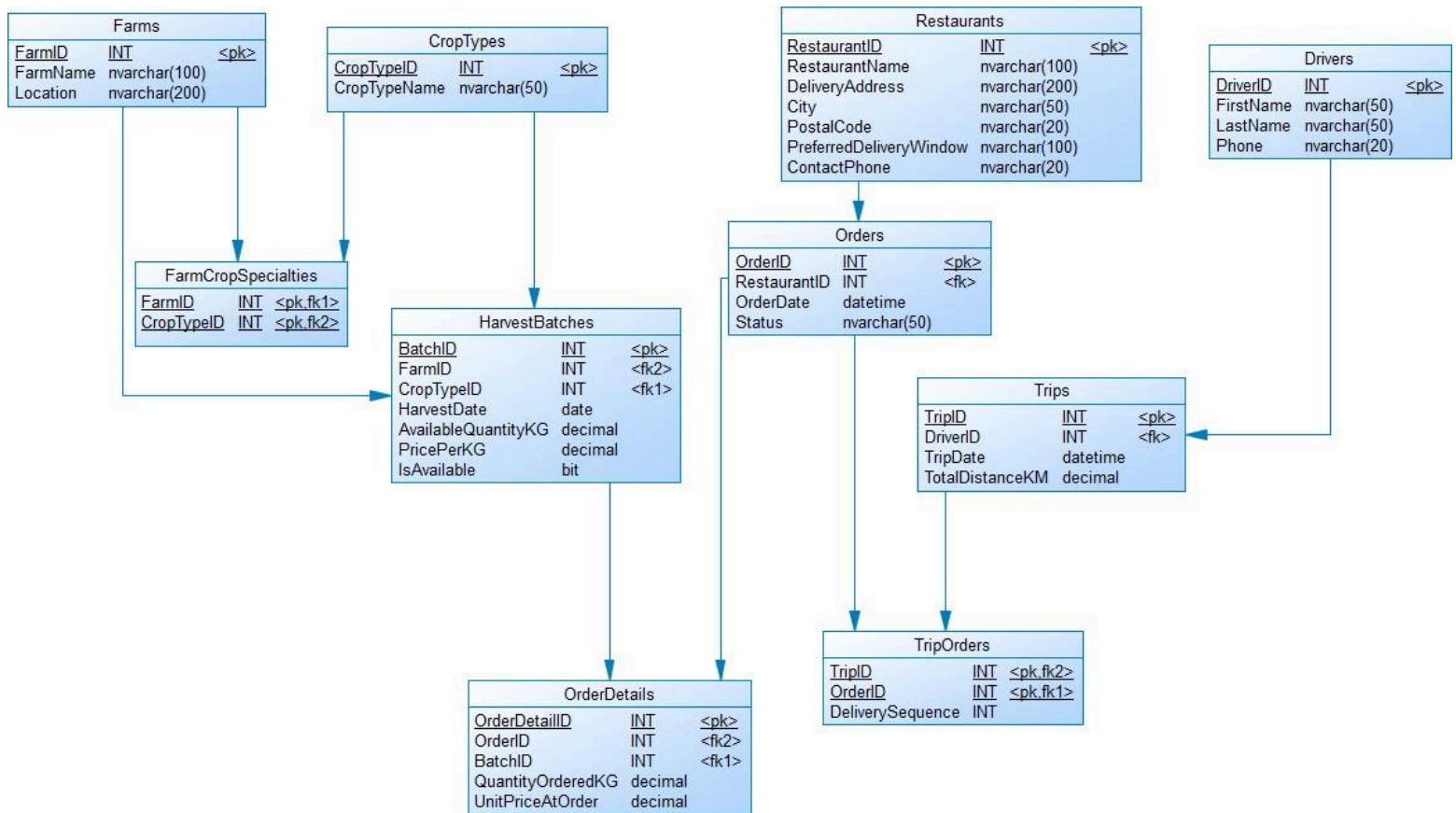
- **Primary Keys (PK):** Strong entities such as Farms, Restaurants, Drivers, HarvestBatches, Orders, and DeliveryTrips were assigned unique surrogate identity keys (e.g., INT IDENTITY) to uniquely identify records.
- **Foreign Keys (FK):** Foreign keys were embedded on the "Many" side of all 1:M relationships to enforce referential integrity. For example, RestaurantID was added to the Orders table, and FarmID was added to the HarvestBatches table.

3. Data Types and Constraints

Abstract attributes from the conceptual model were mapped to concrete MS SQL Server data types.

- String attributes (names, addresses) were mapped to VARCHAR or NVARCHAR with appropriate length limits.
- Numerical values involving measurements or finance (quantities) were mapped to DECIMAL or INT to prevent precision loss.
- Temporal data (harvest dates, delivery windows, order placement times) were strictly typed as DATETIME or DATE.
- Database-level constraints, including NOT NULL for mandatory fields and CHECK constraints (e.g., ensuring quantities are greater than zero), were implemented to guarantee strict data validation before application-level logic occurs.

6. Physical Entity Relationship Diagram (ERD)



7. Graphical User Interface (GUI) Implementation (Bonus)

To fulfill the bonus requirement and provide a user-friendly way to interact with the database, a complete, full-stack web application was developed. The system is deployed live and can be accessed over the internet.

Hosting & Infrastructure:

- **Frontend GUI Web Application:** <https://db.yehia.dev>
- **Database Server:** The MS SQL Server is self-hosted on a virtual private server, accessible at sql.yehia.dev.

The application utilizes a modern tech stack completely **avoiding** the use of Object-Relational Mappers (ORMs), ensuring all SQL statements are written natively in the backend as required.

- **Frontend:** Built using Next.js and React, featuring a responsive, modern interface with a dark-mode toggle and dynamic data tables.
- **Backend:** A Python-based Flask server acts as the API bridge. It accepts HTTP requests from the frontend, executes raw SQL commands directly against the MS SQL Server using the pyodbc driver, and securely returns the JSON data to the client.

GUI Features & Data Integration: The interface provides complete control over the Farm-to-Table database through several dedicated modules:

1. **Dynamic CRUD Operations Panel:** Users can select any entity in the database (e.g., Farms, Restaurants, Orders, Harvest Batches) from a dropdown menu. The GUI dynamically renders a data table for that entity, allowing users to:
 - **Insert** new records directly into the tables.
 - **Update** existing records based on specific conditions (e.g., updating delivery statuses or modifying harvest quantities).
 - **Delete** records securely with required conditional checks.
 - **Select & View** raw data from individual tables in an organized, paginated format.
2. **Advanced Reporting & Joins:** A dedicated "Reports/Inquiries" section allows users to view complex datasets that involve multiple tables. The backend executes predefined JOIN statements to generate business-level insights, such as linking Orders to Restaurants and the corresponding Delivery_Trips to show a comprehensive delivery manifest.
3. **Embedded Documentation:** The GUI includes built-in views to reference the system architecture and the Entity Relationship Diagrams directly within the browser without needing the external documentation.